VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

SCHOOL OF COMPUTER SCIENCE ENGINEERING AND INFORMATION SYSTEMS

WINTER SEMESTER 2024-25

REVIEW-3

COURSE CODE - SWE2009

COURSE - DATA MINING TECHNIQUES

SLOT - E1+TE1

TEAM MEMBERS:

HARIHARAN S -22MIS0340

SAMUEL RAJ A – 22MIS0358

GOWSICK N – 22MIS0523

HARISH S -22MIS0536

Faculty: Prof. DURAI RAJ VINCENT

TITLE OF THE PROJECT: INDOOR WASTE DETECTION

ABSTRACT:

This project focuses on the development of an automatic waste detection system for indoor public places using deep learning techniques. The system is designed to detect and classify four common waste categories: paper, plastic, cap, and shell. The project utilizes a dataset of 4000 color images with a resolution of 1280x720, which was augmented through mirroring and rotation techniques to increase the dataset size to 8000 images. Additionally, synthetic data was generated using Generative Adversarial Networks (GANs) to further enhance the training dataset. Two state-of-the-art object detection models, YOLOv8 and R-CNN, were implemented and evaluated for waste detection performance. The comparative analysis reveals that while R-CNN achieves higher accuracy with a mean Average Precision (mAP@0.5) of 0.6723, YOLOv8 provides significantly faster inference at 41.7 frames per second, making it more suitable for real-time applications.

BRIEF DESCRIPTION OF THE PROJECT:

The project addresses the challenge of automatically detecting and classifying waste in indoor public places using deep learning-based object detection models. The focus is on identifying four common waste categories: paper, plastic, cap, and shell, which represent a significant portion of indoor waste materials. The improper management of waste in indoor public places poses significant environmental and health challenges. Manual waste detection and sorting are labor-intensive, time-consuming, and often inefficient. Automating this process through computer vision and deep learning techniques can significantly improve waste management efficiency, reduce human labor costs, and contribute to better recycling practices.

OUTCOME OF THE PROJECT:

The project successfully implemented and compared two state-of-the-art object detection models for automatic waste detection. Key outcomes include:

1. Both models demonstrate strong performance in detecting and classifying four common waste categories

2. R-CNN achieves higher detection accuracy (mAP@0.5 of 0.6723) but at the cost of slower inference speed (5.3 FPS)

3. YOLOv8 provides significantly faster detection (41.7 FPS) while maintaining good accuracy (mAP@0.5 of 0.6104), making it more suitable for real-time applications

4. Data augmentation and synthetic data generation using GANs significantly improved model performance

5. The project demonstrates the effectiveness of deep learning approaches for waste detection and provides insights into the trade-offs between detection accuracy and processing speed for practical deployment in indoor public environments

LITERATURE REVIEW :

S. NO.	TITLE	YEAR	METHODOLOGY	RESULT	RESEARCH GAP
1	Skip-YOLO: Domestic Garbage Detection Using Deep Learning Method in Complex Multi-scene	2023	Proposed Skip-YOLO model with dense convolution module to improve backbone network for better feature mapping, enhancing high-dimensional feature extraction and reducing overfitting. The model was trained on a domestic garbage dataset with 304 single-class and 914 multi-class images, split into 80% training, 10% validation, and 10% testing.	Established a domestic garbage dataset and showed improved accuracy through comparative experiments, with a focus on indoor and outdoor life scenes, including dormitory and black garbage bag backgrounds.	Difficulty in recognizing domestic garbage with similar characteristics in complex scenes, particularly in indoor settings with varied lighting and backgrounds.
2	An MRS-YOLO Model for High-Precision Waste Detection and Classification	2024	Introduced SlideLoss_IOU for small object detection, integrated RepViT of the Transformer mechanism, and devised a novel	Improved mAP50% accuracy by 3.6% compared to YOLOv8, reduced model volume	Need for high-precision waste detection and classification, especially for small and diverse indoor waste items.

			feature extraction strategy using multi-dimensional and dynamic convolution mechanisms. Validated on a dataset of 12,072 samples across 10 categories, including recyclable metal and paper, with indoor images taken in buildings.	by 15.09%, and demonstrated enhanced accuracy in detecting small targets, with comprehensive detection capabilities across diverse scenarios, including home backgrounds.	
3	Deep learning-based waste detection in natural and urban environments	2022	Conducted a survey and created new benchmark datasets (detect-waste and classify-waste) by merging existing open-source datasets with unified annotations for categories like bio, glass, metal, plastic, non-recyclable, paper, and unknown. Used deep learning models for waste detection analysis.	Provided a baseline for litter detection, summarizing previous studies and offering experimental results on the presented datasets, applicable to both indoor and outdoor urban settings.	Lack of benchmarks and standards for waste detection, particularly for indoor environments with diverse waste types.
4	Intelligent waste management system using deep learning with IoT	2020	Used a convolutional neural network (CNN) for waste classification into digestible and indigestible categories,	Achieved high accuracy in waste label classification, with sensors estimating data and system	Need for intelligent waste management systems combining deep learning and IoT, especially for real-time indoor waste monitoring.

			integrated with IoT for real-time monitoring using a smart trash bin with sensors and Bluetooth connectivity for data monitoring via an android application. 2	usability scale (SUS) evaluated, suitable for indoor settings like homes and offices.	
5	A deep learning approach based hardware solution to categorise garbage in environment	2021	Developed SmartBin using four pre-trained CNNs (AlexNet, ResNet, VGG-16, InceptionNet), with InceptionNet performing best (accuracy 96.23–98.15%, loss 0.10–0.13). Images captured via pi-cam and processed on Raspberry Pi for classification, with servo motor for segregation.	Segregated garbage into biodegradable and non-biodegradable categories, providing a low-cost solution for base-level garbage detection and segregation, applicable to indoor environments.	Need for hardware solutions for garbage categorization, particularly for indoor settings with limited resources.
6	Garbage detection and classification using a new deep learning-based machine vision system as a tool for sustainable waste recycling	2023	Proposed an improved MobileNetV2 with attention mechanism in first and last convolution layers, using transfer learning with pre-trained weights and PCA for dimension reduction, enabling real-time operation on edge devices.	Achieved real-time garbage detection and classification, improving recognition accuracy and model generalization , suitable for indoor waste sorting applications.	Need for efficient and accurate machine vision systems for waste recycling, especially in indoor settings with diverse waste.

7	Real time electronic-waste classification algorithms using the computer vision based on Convolutional Neural Network (CNN): Enhanced environmental	2024	Trained a computer vision model using CNN for e-waste component material identification, utilizing YOLO 7 and 5 with TensorFlow 2.8.0, evaluated on 240 e-waste objects.	Demonstrated ~94% correct prediction for batch size of 16, potential for customer assistance and robotic platforms in indoor settings like recycling facilities.	Lack of accessible information on e-waste for training models, hindering indoor e-waste detection accuracy.
8	Recyclable waste image recognition based on deep learning	2021	Used deep learning for recyclable waste image recognition, focusing on sorting materials through image recognition, predictive modeling, anomaly detection, and decision-making for cost-effective options.	Improved waste sorting and recycling efficiency, applicable to indoor waste management systems for households and offices.	Need for automated solutions due to increasing waste volumes, particularly in indoor environments.
9	WasteInNet: Deep Learning Model for Real-time Identification of Various Types of Waste	2024	Developed a deep learning model for real-time waste detection and classification, emphasizing visual inspection and computer vision algorithms for waste categorization by type.	Emphasized accurate waste identification for efficient management, suitable for indoor settings with real-time monitoring needs.	Challenges in real-time monitoring and automated sorting systems, especially for indoor waste with diverse categories.

10	Intelligent Waste Classification System Using Deep Learning Convolutional Neural Network	2019	Used ResNet-50 as a feature extractor and Support Vector Machine (SVM) for classifying waste into groups like glass, metal, paper, and plastic, focusing on simplifying manual hand-picking processes.	Simplified waste material classification process, applicable to indoor waste management for urban areas.	Need for advanced/intelligent waste management systems, particularly for indoor settings with varied waste types.
11	A real-time rural domestic garbage detection algorithm with an improved YOLOv5s network model	2022	Improved YOLOv5s-CSS model with attention combination mechanism, added small object detection layer, adopted CIoU loss function, and used Adam optimization, trained on 7671 samples across 13 categories.	Achieved precision 93.5%, recall 91.1%, mAP@0.5 96.4%, FPS 47.6, inference time 0.021s, suitable for domestic indoor waste detection in rural settings.	Need for fast and accurate detection of small targets in complex backgrounds, particularly in indoor domestic environments.
12	YOLO Based Model for Garbage Detection	2022	Used YOLO for garbage detection in real-time images, focusing on collecting pictures of garbage on roads or in bodies of water, with the YOLO frame working on Darknet for visual perception.	Enabled identification and monitoring of garbage, with potential application to indoor settings for real-time waste detection.	Need for automated garbage detection to reduce manual monitoring time, especially in indoor environments.
13	An automated	2022	Optimized YOLO	Developed a	Need for accurate

	solid waste detection using the optimized YOLO model for riverine management		model for detecting floating garbage in rivers, focusing on visual information acquisition under fluctuating illumination, complex background, and occlusion, for robotic platforms.	system for river cleaning robots, with potential insights for indoor waste detection systems.	computer vision systems for robotic platforms, applicable to indoor waste management.
14	EcoDetect-YOLO: A Lightweight, High-Generalizatio n Methodology for Real-Time Detection of Domestic Waste Exposure in Intricate Environmenta l Landscapes	2024	Proposed EcoDetect-YOLO algorithm based on YOLOv5s, integrated CBAM between P2 and P3 layers, added P2 small-target detection head, and introduced BiFPN for deep feature fusion, using a new dataset.	Improved detection accuracy and generalization for domestic waste in complex environments, with applications in indoor settings.	Need for lightweight, high-performance models for real-time waste detection in intricate indoor settings.
15	A multi-label waste detection model based on transfer learning	2023	Proposed YOLO-WASTE model for multi-label waste classification, using transfer learning to speed up learning efficiency, with a dataset including multiple wastes per image.	Improved accuracy and efficiency in waste detection, suitable for indoor multi-label waste classification.	Need for multi-label waste classification models, particularly for indoor environments with diverse waste.
16	Smart e-waste management system utilizing	2023	Used YOLOv5s, YOLOv7-tiny, and YOLOv8s for e-waste object	Efficient and accurate e-waste detection and	Need for innovative, safer, and greener e-waste management

	Internet of Things and Deep Learning approaches		detection, integrated with IoT for real-time monitoring, using TensorFlow Lite for edge deployment.	management, applicable to indoor recycling facilities.	systems, especially for indoor settings.
17	Deep learning networks for real-time regional domestic waste detection	2022	Used Yolo-v3 for domestic waste detection, expanded Taiwan recycled waste dataset (TRWD) for training, compared with TrashNet, focusing on multiple waste objects.	Improved recognition performance of household waste products, with potential for indoor domestic waste detection.	Need for enhanced recognition performance in waste sorting, particularly in indoor settings with multiple waste types.
18	Garbage Detection Algorithm Based on YOLO v3	2022	Introduced Efficient Channel Attention YOLO (ECA-YOLO) with attention mechanism in residual units, trained on Huawei Garbage Classification Challenge Cup dataset.	Improved detection mAP by 1.07%, high accuracy and recall rate, with potential for indoor garbage detection.	Need for high accuracy and recall in garbage detection, especially in indoor complex scenes.
19	YOLO TrashNet: Garbage Detection in Video Streams	2020	Improved YOLOv3 network model for garbage detection in video streams, fine-tuned on a dataset collected for abandoned waste detection, focusing on real-time analysis.	Effective for smart waste management in smart cities, with applications in indoor video stream monitoring.	Need for real-time garbage detection in video streams, particularly for indoor settings.

20	Solid Waste Detection Using Enhanced YOLOv8 Lightweight Convolutional Neural Networks	2024	Improved YOLOv8s with CG-HGNetV2 backbone, CBAM attention module, P2 small-target detection head, and BiFPN for feature fusion, focusing on lightweight and high-performance detection.	Enhanced detection accuracy and efficiency for solid waste in complex urban environments, suitable for indoor settings.	Need for lightweight and high-performance models for real-time waste detection, especially in indoor urban contexts.

PROPOSED WORK/MODEL:

The project implemented two main models for waste detection:

1. YOLOv8 Implementation:

- Architecture: Uses a CSPDarknet53 backbone for feature extraction, Path Aggregation Network (PAN) for feature fusion, and detection heads for predicting object locations
- Training Configuration: Input size 640x640 pixels, batch size 16, learning rate 0.01 with cosine annealing scheduler, SGD optimizer, 100 training epochs
- Implementation used the Ultralytics YOLOv8 framework with pre-trained weights from COCO dataset

2. R-CNN Implementation:

- Architecture: Faster R-CNN with ResNet-50 backbone, Region Proposal Network, RoI pooling layer, and classification/bounding box regression heads
- Training Configuration: Original 1280x720 pixel input, batch size 4, learning rate 0.005 with step decay, SGD optimizer, 50 training epochs
- Implementation used PyTorch torchvision library with pre-trained weights from COCO dataset

3. Data Enhancement:

- Data Augmentation: Mirroring, rotation, brightness/contrast adjustments, and random cropping to increase dataset from 4000 to 8000 images
- Synthetic Data Generation: GAN implementation with generator and discriminator networks to create additional training examples

SOURCE CODE :

YOLO V8:

```
!pip install ultralytics
 !pip install ray==2.0.0
# 2. Patch Ray to Avoid _get_session Error
try:
    import ray
    try:
        import ray.train._internal.session as session
        if not hasattr(session, "_get_session"):
            session._get_session = lambda: None
            print("Ray patch applied: _get_session has been set to a dummy function.")
        else:
            print("ray.train._internal.session already has _get_session.")
    except Exception as inner_error:
        print("Could not patch ray.train._internal.session:", inner_error)
except ImportError:
    print("Ray is not installed. Skipping ray patching.")
# 3. Imports and Directory Setup
import os
import shutil
import torch
import yaml
from pathlib import Path
from ultralytics import YOLO
```

```
# 3A) Define paths for original dataset

# NOTE: Adjust these paths based on where your original dataset is located.

ORIG_TRAIN_IMAGES = "/kaggle/input/wastet/train/images"

ORIG_TRAIN_LABELS = "/kaggle/input/wastet/train/labels"

VAL_IMAGES = "/kaggle/input/wastet/valid/images"

TEST_IMAGES = "/kaggle/input/wastet/test/images"

# 3B) Define where your downloaded GAN images are located

# Suppose you downloaded them via Kaggle CLI to a folder named "kaggle_output" in
the current directory:

GAN_ROOT = "/kaggle/input/wasten/generated_images"

# 3C) Define folders where the merged (augmented) dataset will go

AUG_IMAGES = "augmented_train/images"

AUG_LABELS = "augmented_train/labels"

os.makedirs(AUG_IMAGES, exist_ok=True)

os.makedirs(AUG_LABELS, exist_ok=True)

# Number of classes in your dataset (adjust if needed)

num_classes = 4

class_names = ['cap', 'paper', 'plastic', 'shell']  # Adjust to match your actual class labels

# 4. Merge Original with GAN Images

# 4A) Copy original training images + labels

print("Copying original training images and labels...")

for file in os.listdir(ORIG_TRAIN_IMAGES):

    if file.lower().endswith('.jpg'):

        shutil.copy(os.path.join(ORIG_TRAIN_IMAGES, file),
os.path.join(AUG_IMAGES, file))

for file in os.listdir(ORIG_TRAIN_LABELS):
```

```
    if file.lower().endswith('.txt'):
        shutil.copy(os.path.join(ORIG_TRAIN_LABELS, file),
os.path.join(AUG_LABELS, file))
# 4B) Add the GAN-generated images (PNG)
#     Each subfolder (0,1,2,3) represents a class. We'll assign a YOLO label covering the
entire image.
print("Adding GAN generated images to the dataset (assigning full-image bounding
boxes)...")
for class_id in range(num_classes):
    class_folder = os.path.join(GAN_ROOT, str(class_id))
    if not os.path.exists(class_folder):
        print(f"Warning: Folder for class {class_id} not found at {class_folder}. Skipping.")
        continue
    for file in os.listdir(class_folder):
        if file.lower().endswith('.png'):
            # Create a unique image name (avoid conflicts with original dataset)
            new_img_name = f"gan_{class_id}_{file}"
            new_label_name = new_img_name.rsplit('.', 1)[0] + '.txt'
           src_img_path = os.path.join(class_folder, file)
            dst_img_path = os.path.join(AUG_IMAGES, new_img_name)
            shutil.copy(src_img_path, dst_img_path)
            # YOLO format: <class> <x_center> <y_center> <width> <height>
            # We'll assume that the synthetic image is fully occupied by the object.
            annotation_line = f"{class_id} 0.5 0.5 1.0 1.0\n"
            with open(os.path.join(AUG_LABELS, new_label_name), 'w') as label_file:
                label_file.write(annotation_line)
```

```
print("Dataset merging completed.")
# 5. Create or Update data.yaml
# The new data.yaml points to the augmented dataset for training,
# while the validation and test remain the same.
data_yaml = {
    'train': str(Path(AUG_IMAGES).absolute()),
    'val': VAL_IMAGES,
    'test': TEST_IMAGES,
    'nc': num_classes,
    'names': class_names
}
data_yaml_path = "data.yaml"
with open(data_yaml_path, 'w') as f:
    yaml.dump(data_yaml, f)
print("\nGenerated data.yaml configuration:")
with open(data_yaml_path, 'r') as f:
    print(f.read())
# 6. Verify Paths and Check GPU
print("Verifying dataset paths:")
print("Train path exists:", os.path.exists(data_yaml['train']))
print("Validation path exists:", os.path.exists(data_yaml['val']))
print("Test path exists:", os.path.exists(data_yaml['test']))
print("\nGPU Available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU Name:", torch.cuda.get_device_name(0))
```

```
# 7. Train YOLOv8 Model on Merged Dataset
# Initialize YOLOv8 with a pretrained checkpoint (e.g. yolov8n.pt)
model = YOLO('yolov8n.pt')
train_args = {
    'data': str(Path(data_yaml_path).absolute()),
    'epochs': 100,
    'imgsz': 640,
    'batch': 8,
    'patience': 20,
    'device': 0,            # Use GPU if available
    'workers': 2,
    'save': True,
    'cache': True,
    'project': 'waste_detection_augmented',
    'name': 'training_run_augmented',
    'exist_ok': True
}
print("\nStarting YOLOv8 training on augmented dataset...")
model.train(**train_args)
print("Training completed successfully!")
# 8. Save the Trained Model
model_save_path = 'waste_detection_augmented_best.pt'
model.save(model_save_path)
print(f"\nModel saved successfully as {model_save_path}")
# Verify that the file was saved
```

```
print("\nSaved model file details:")
!ls -lh {model_save_path}
```

R-CNN :

```
import os
import yaml
import torch
import torchvision
from torchvision.models.detection import fasterrcnn_resnet50_fpn
import torchvision.transforms as transforms
from torch.utils.data import Dataset, DataLoader
from PIL import Image
# 1. Data Configuration
DATASET_ROOT = "/content/my_dataset"  # Adjust as needed
data_config = {
    'train': {
        'images': os.path.join(DATASET_ROOT, 'train', 'images'),
        'labels': os.path.join(DATASET_ROOT, 'train', 'labels')
    },
    'val': {
        'images': os.path.join(DATASET_ROOT, 'valid', 'images'),
        'labels': os.path.join(DATASET_ROOT, 'valid', 'labels')
    },
    'test': {
        'images': os.path.join(DATASET_ROOT, 'test', 'images'),
        'labels': os.path.join(DATASET_ROOT, 'test', 'labels')
```

```
    },
    'nc': 4,  # number of classes
    'names': ['cap', 'paper', 'plastic', 'shell'],
    'roboflow': {
        'workspace': 'hari-oetwc',
        'project': 'indoor-waste-detection',
        'version': 5,
        'license': 'CC BY 4.0',
        'url': 'https://universe.roboflow.com/hari-oetwc/indoor-waste-detection/dataset/5'
    }
}
with open('data.yaml', 'w') as f:
    yaml.dump(data_config, f)
print("Data configuration saved in data.yaml.")
# 2. Verify Dataset Paths and GPU
print("\nVerifying dataset paths:")
print("Train images exist:", os.path.exists(data_config['train']['images']))
print("Train labels exist:", os.path.exists(data_config['train']['labels']))
print("Valid images exist:", os.path.exists(data_config['val']['images']))
print("Valid labels exist:", os.path.exists(data_config['val']['labels']))
print("Test images exist:", os.path.exists(data_config['test']['images']))
print("Test labels exist:", os.path.exists(data_config['test']['labels']))
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
print("\nUsing device:", device)
if device.type == 'cuda':
```

```
    print("GPU Name:", torch.cuda.get_device_name(0))
# 3. Custom Dataset for Faster R-CNN
class WasteDataset(Dataset):
    """
    Custom dataset for indoor waste detection.
    Images and labels are stored in separate folders.
    The annotations are in YOLO format (normalized):
        <class> <x_center> <y_center> <width> <height>
    This version pre-parses the label files to filter out images with no valid boxes.
    """
    def __init__(self, images_dir, labels_dir, transform=None):
        self.images_dir = images_dir
        self.labels_dir = labels_dir
        self.transform = transform
        self.image_files = []

        # Loop through all candidate image files
        for f in os.listdir(images_dir):
            if f.lower().endswith(('.jpg', '.jpeg', '.png')):
                base_name = os.path.splitext(f)[0]
                label_path = os.path.join(labels_dir, base_name + ".txt")
                # If the label file exists and is non-empty, try to parse it
                if os.path.exists(label_path) and os.path.getsize(label_path) > 0:
                    valid_boxes = self._parse_label_file(label_path,
image_path=os.path.join(images_dir, f))
```

```
                if valid_boxes is not None and len(valid_boxes) > 0:
                    self.image_files.append(f)
        if len(self.image_files) == 0:
            raise ValueError("No images with valid annotations found!")
        self.image_files = sorted(self.image_files)
    def _parse_label_file(self, label_path, image_path):
        """Parse a label file and return list of boxes. Returns empty list if none."""
        try:
            img = Image.open(image_path)
        except Exception as e:
            return []
        w, h = img.size
        boxes = []
        with open(label_path, 'r') as f:
            for line in f:
                parts = line.strip().split()
                if len(parts) != 5:
                    continue
                try:
                    cls, x_center, y_center, w_norm, h_norm = map(float, parts)
                except Exception:
                    continue
                # Convert normalized coordinates to absolute
                x_center_abs = x_center * w
                y_center_abs = y_center * h
```

```
            width_abs = w_norm * w
            height_abs = h_norm * h
            xmin = x_center_abs - width_abs / 2
            ymin = y_center_abs - height_abs / 2
            xmax = x_center_abs + width_abs / 2
            ymax = y_center_abs + height_abs / 2
            boxes.append([xmin, ymin, xmax, ymax])
    return boxes
def __len__(self):
    return len(self.image_files)
def __getitem__(self, idx):
    image_filename = self.image_files[idx]
    image_path = os.path.join(self.images_dir, image_filename)
    img = Image.open(image_path).convert("RGB")
    w, h = img.size
    base_name = os.path.splitext(image_filename)[0]
    label_path = os.path.join(self.labels_dir, base_name + ".txt")
    boxes = []
    labels = []
    with open(label_path, 'r') as f:
        for line in f:
            parts = line.strip().split()
            if len(parts) != 5:
                continue
            cls, x_center, y_center, w_norm, h_norm = map(float, parts)
```

```
            # Convert normalized coordinates to absolute coordinates
            x_center_abs = x_center * w
            y_center_abs = y_center * h
            width_abs = w_norm * w
            height_abs = h_norm * h
            xmin = x_center_abs - width_abs / 2
            ymin = y_center_abs - height_abs / 2
            xmax = x_center_abs + width_abs / 2
            ymax = y_center_abs + height_abs / 2
            boxes.append([xmin, ymin, xmax, ymax])
            labels.append(int(cls) + 1)  # 1-indexed for Faster R-CNN

    # Make sure there is at least one valid box
    if len(boxes) == 0:
        raise ValueError(f"No valid annotations for image {image_filename}")
    boxes = torch.as_tensor(boxes, dtype=torch.float32)
    labels = torch.as_tensor(labels, dtype=torch.int64)

    target = {
        "boxes": boxes,
        "labels": labels,
        "image_id": torch.tensor([idx])
    }
    if self.transform:
        img = self.transform(img)
```

```
        else:
            img = transforms.ToTensor()(img)
        return img, target
transform = transforms.Compose([transforms.ToTensor()])
# Create datasets and dataloaders
train_dataset = WasteDataset(
    images_dir=data_config['train']['images'],
    labels_dir=data_config['train']['labels'],
    transform=transform
)
val_dataset = WasteDataset(
    images_dir=data_config['val']['images'],
    labels_dir=data_config['val']['labels'],
    transform=transform
)
def collate_fn(batch):
    return tuple(zip(*batch))
train_loader = DataLoader(train_dataset, batch_size=4, shuffle=True,
collate_fn=collate_fn)
val_loader = DataLoader(val_dataset, batch_size=4, shuffle=False, collate_fn=collate_fn)
# 4. Initialize and Train Faster R-CNN
fasterrcnn_model = fasterrcnn_resnet50_fpn(pretrained=True, progress=True)
num_classes = data_config['nc'] + 1  # 4 classes + background
in_features = fasterrcnn_model.roi_heads.box_predictor.cls_score.in_features
fasterrcnn_model.roi_heads.box_predictor =
torchvision.models.detection.faster_rcnn.FastRCNNPredictor(
```

```
    in_features, num_classes
)
fasterrcnn_model.to(device)
params = [p for p in fasterrcnn_model.parameters() if p.requires_grad]
optimizer = torch.optim.SGD(params, lr=0.005, momentum=0.9, weight_decay=0.0005)
def train_fasterrcnn(model, data_loader, optimizer, device, num_epochs=4):
    model.train()
    for epoch in range(num_epochs):
        epoch_loss = 0.0
        for images, targets in data_loader:
            images = list(img.to(device) for img in images)
            targets = [{k: v.to(device) for k, v in t.items()} for t in targets]
            loss_dict = model(images, targets)
            losses = sum(loss for loss in loss_dict.values())
            optimizer.zero_grad()
            losses.backward()
            optimizer.step()
            epoch_loss += losses.item()
        avg_loss = epoch_loss / len(data_loader)
        print(f"Epoch [{epoch+1}/{num_epochs}] - Loss: {avg_loss:.4f}")
print("\nStarting Faster R-CNN training...")
train_fasterrcnn(fasterrcnn_model, train_loader, optimizer, device, num_epochs=7)
print("Faster R-CNN training completed successfully!")
# 5. Evaluate Faster R-CNN
def evaluate_fasterrcnn(model, data_loader, device):
```

```
    model.eval()
    total_loss = 0.0
    with torch.no_grad():
        for images, targets in data_loader:
            images = list(img.to(device) for img in images)
            targets = [{k: v.to(device) for k, v in t.items()} for t in targets]
            loss_dict = model(images, targets)
            losses = sum(loss for loss in loss_dict.values())
            total_loss += losses.item()
    avg_loss = total_loss / len(data_loader)
    print(f"\nFaster R-CNN - Average Validation Loss: {avg_loss:.4f}")
    return avg_loss
val_loss = evaluate_fasterrcnn(fasterrcnn_model, val_loader, device)
# 6. Save the Trained Model
model_save_path = "waste_detection_fasterrcnn.pth"
torch.save(fasterrcnn_model.state_dict(), model_save_path)
print(f"\nFaster R-CNN model saved to: {model_save_path}")
# 7. Summary
print("\nTraining and Evaluation Summary:")
print(f"- Final Average Validation Loss: {val_loss:.4f}")
```

GAN :

```
import os
import cv2
import numpy as np
```

```
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, Subset
from torchvision.datasets import ImageFolder
 Step 1: Extract Cropped Images of Each Object
def extract_crops(image_dir, annotation_dir, output_dir):
    """
    Extract cropped images from a dataset based on bounding box annotations and organize them by class.
    Args:
        image_dir (str): Directory containing original images.
        annotation_dir (str): Directory containing annotation files in YOLO format.
        output_dir (str): Directory to save cropped images, organized by class.
    """
    os.makedirs(output_dir, exist_ok=True)
    for img_file in os.listdir(image_dir):
        if not img_file.endswith('.jpg'):
            continue
        img_path = os.path.join(image_dir, img_file)
        ann_path = os.path.join(annotation_dir, img_file.replace('.jpg', '.txt'))
        if not os.path.exists(ann_path):
            continue
        img = cv2.imread(img_path)
```

```
        height, width, _ = img.shape
        with open(ann_path, 'r') as f:
            for line in f:
                parts = line.strip().split()
                class_id = int(parts[0])
                x_center = float(parts[1]) * width
                y_center = float(parts[2]) * height
                bbox_width = float(parts[3]) * width
                bbox_height = float(parts[4]) * height
                x1 = int(x_center - bbox_width / 2)
                y1 = int(y_center - bbox_height / 2)
                x2 = int(x_center + bbox_width / 2)
                y2 = int(y_center + bbox_height / 2)
                x1, y1 = max(0, x1), max(0, y1)
                x2, y2 = min(width, x2), min(height, y2)
                if x2 <= x1 or y2 <= y1:
                    continue
                crop = img[y1:y2, x1:x2]
                crop_resized = cv2.resize(crop, (128, 128))
                class_dir = os.path.join(output_dir, str(class_id))
                os.makedirs(class_dir, exist_ok=True)
                crop_filename = f"{img_file.split('.')[0]}_{x1}_{y1}.jpg"
                cv2.imwrite(os.path.join(class_dir, crop_filename), crop_resized)
# Run for training set
```

```
extract_crops('/kaggle/input/wastet/train/images', '/kaggle/input/wastet/train/labels',
'cropped_images')

# ### Step 2: Define GAN Architecture

image_size = 128

latent_dim = 100

num_epochs = 200

batch_size = 64

learning_rate = 0.0002

beta1 = 0.5

class Generator(nn.Module):

    def __init__(self):

        super(Generator, self).__init__()

        self.main = nn.Sequential(

            nn.ConvTranspose2d(latent_dim, 512, 4, 1, 0, bias=False),

            nn.BatchNorm2d(512),

            nn.ReLU(True),

            nn.ConvTranspose2d(512, 256, 4, 2, 1, bias=False),

            nn.BatchNorm2d(256),

            nn.ReLU(True),

            nn.ConvTranspose2d(256, 128, 4, 2, 1, bias=False),

            nn.BatchNorm2d(128),

            nn.ReLU(True),

            nn.ConvTranspose2d(128, 64, 4, 2, 1, bias=False),

            nn.BatchNorm2d(64),

            nn.ReLU(True),
```

```
 nn.ConvTranspose2d(64, 32, 4, 2, 1, bias=False),  # New layer
            nn.BatchNorm2d(32),
            nn.ReLU(True),
            nn.ConvTranspose2d(32, 3, 4, 2, 1, bias=False),  # Adjusted output
            nn.Tanh()
        )
    def forward(self, x):
        return self.main(x)
class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()
        self.main = nn.Sequential(
            nn.Conv2d(3, 64, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(64, 128, 4, 2, 1, bias=False),
            nn.BatchNorm2d(128),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(128, 256, 4, 2, 1, bias=False),
            nn.BatchNorm2d(256),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(256, 512, 4, 2, 1, bias=False),
            nn.BatchNorm2d(512),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(512, 1, 8, 1, 0, bias=False),  # Changed kernel size to 8
            nn.Sigmoid()
```

```
        )
    def forward(self, x):
        return self.main(x).view(-1, 1)  # Reshape to [batch_size, 1]
# Step 3: Train GAN for Each Class
def train_gan(class_id, batch_size):
    """
    Train a GAN for a specific class using only images from that class.
    Args:
        class_id (int): The class ID to train the GAN for.
        batch_size (int): The batch size for training.
    """
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    # Define transformations for the images
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
    ])
    # Load the entire dataset
    dataset = ImageFolder(root='/kaggle/working/cropped_images', transform=transform)
    # Get indices of samples belonging to the specified class_id
    indices = [i for i in range(len(dataset)) if dataset.targets[i] == class_id]
    # Create a subset for the specific class
    subset = Subset(dataset, indices)
    # Create a DataLoader for the subset
    dataloader = DataLoader(subset, batch_size=batch_size, shuffle=True)
```

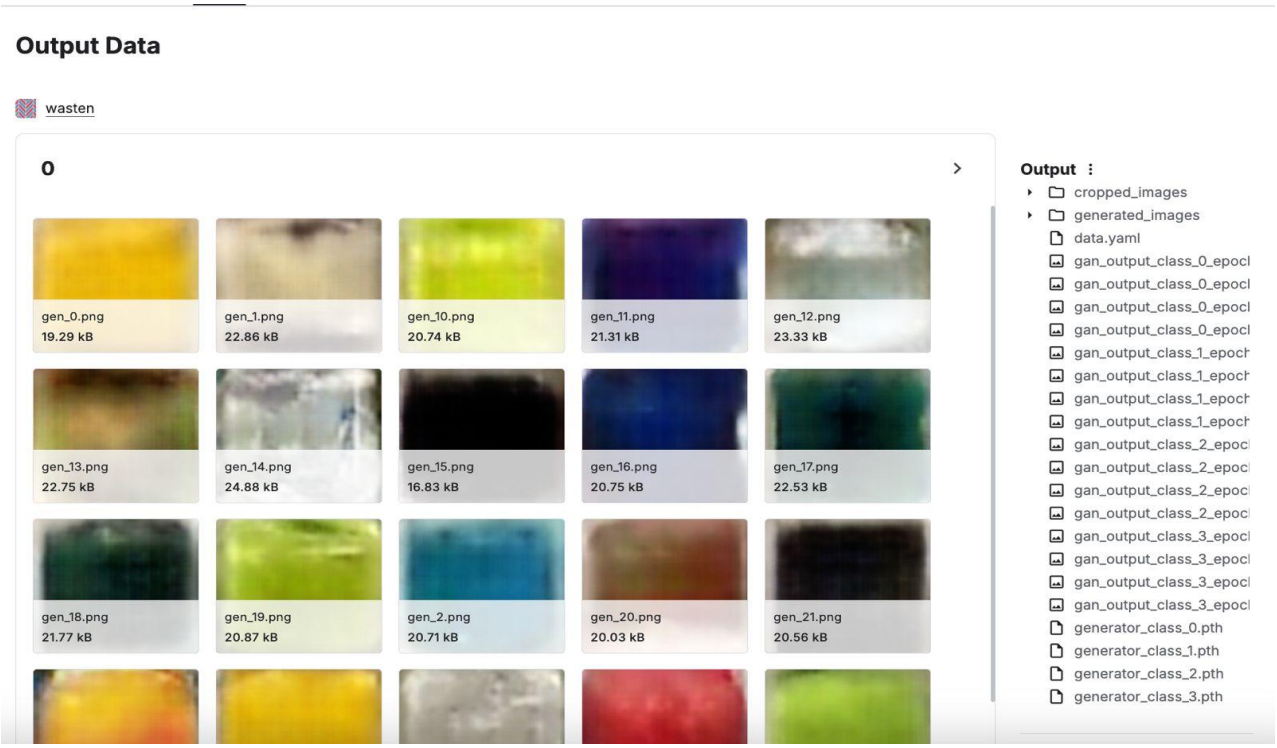
```
    # Initialize networks
    generator = Generator().to(device)
discriminator = Discriminator().to(device)
    # Loss function and optimizers
    criterion = nn.BCELoss()
    optimizer_g = torch.optim.Adam(generator.parameters(), lr=learning_rate,
betas=(beta1, 0.999))
    optimizer_d = torch.optim.Adam(discriminator.parameters(), lr=learning_rate,
betas=(beta1, 0.999))
    # Training loop
    for epoch in range(num_epochs):
        for i, (imgs, _) in enumerate(dataloader):
            real_imgs = imgs.to(device)
            batch_size = real_imgs.size(0)
            # Labels for real and fake images
            real_label = torch.ones(batch_size, 1).to(device)
            fake_label = torch.zeros(batch_size, 1).to(device)
            # Train Discriminator
            optimizer_d.zero_grad()
            outputs = discriminator(real_imgs)
            d_loss_real = criterion(outputs, real_label)
            noise = torch.randn(batch_size, latent_dim, 1, 1).to(device)
            fake_imgs = generator(noise)
            outputs = discriminator(fake_imgs.detach())
            d_loss_fake = criterion(outputs, fake_label)
            d_loss = d_loss_real + d_loss_fake
```

```
            d_loss.backward()
            optimizer_d.step()
            # Train Generator
            optimizer_g.zero_grad()
            outputs = discriminator(fake_imgs)
            g_loss = criterion(outputs, real_label)
            g_loss.backward()
            optimizer_g.step()
        print(f"Class {class_id}, Epoch [{epoch}/{num_epochs}], D Loss: {d_loss.item()}, G Loss: {g_loss.item()}")
        # Save generated images periodically
        if epoch % 50 == 0:
            torchvision.utils.save_image(fake_imgs, f"gan_output_class_{class_id}_epoch_{epoch}.png", normalize=True)
    # Save the trained generator
    torch.save(generator.state_dict(), f"generator_class_{class_id}.pth")
# Train GAN for each class (assuming 4 classes: 0, 1, 2, 3)
for class_id in range(4):
    train_gan(class_id, batch_size)
# Step 4: Generate Synthetic Object Images
def generate_images(class_id, num_images):
    """
    Generate synthetic images using the trained generator for a specific class.
    Args:
        class_id (int): The class ID of the generator to use.
        num_images (int): Number of synthetic images to generate.
```

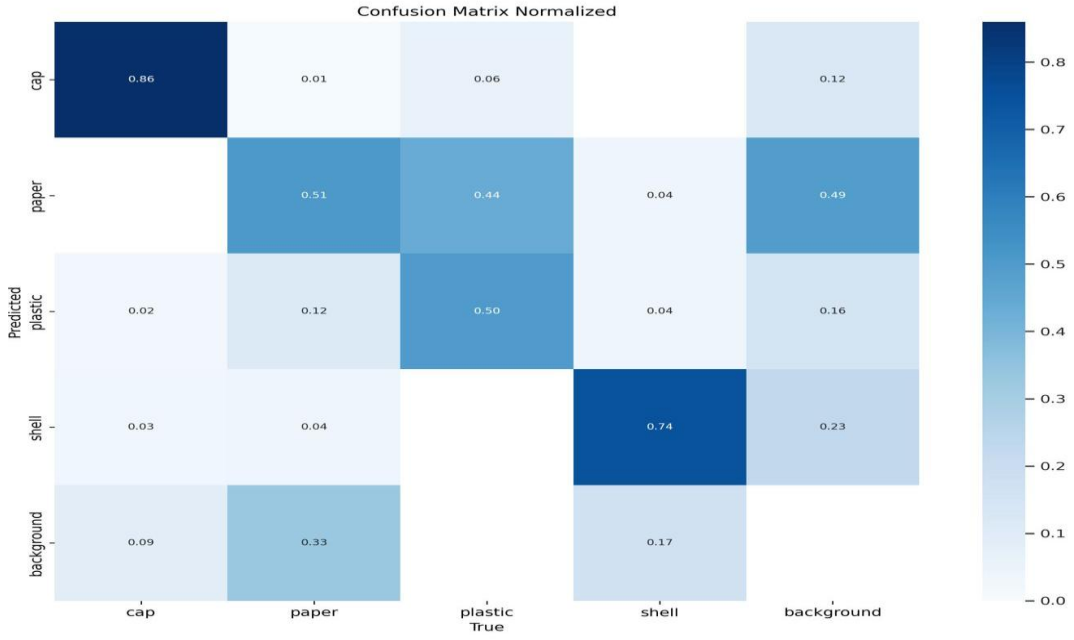
```
    """
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    generator = Generator().to(device)
    generator.load_state_dict(torch.load(f"generator_class_{class_id}.pth"))
    generator.eval()
    os.makedirs(f"generated_images/{class_id}", exist_ok=True)
    with torch.no_grad():
        for i in range(num_images):
            noise = torch.randn(1, latent_dim, 1, 1).to(device)
            fake_img = generator(noise)
            torchvision.utils.save_image(fake_img,
f"generated_images/{class_id}/gen_{i}.png", normalize=True)
# Generate 100 images per class
for class_id in range(4):
    generate_images(class_id, 100)
```

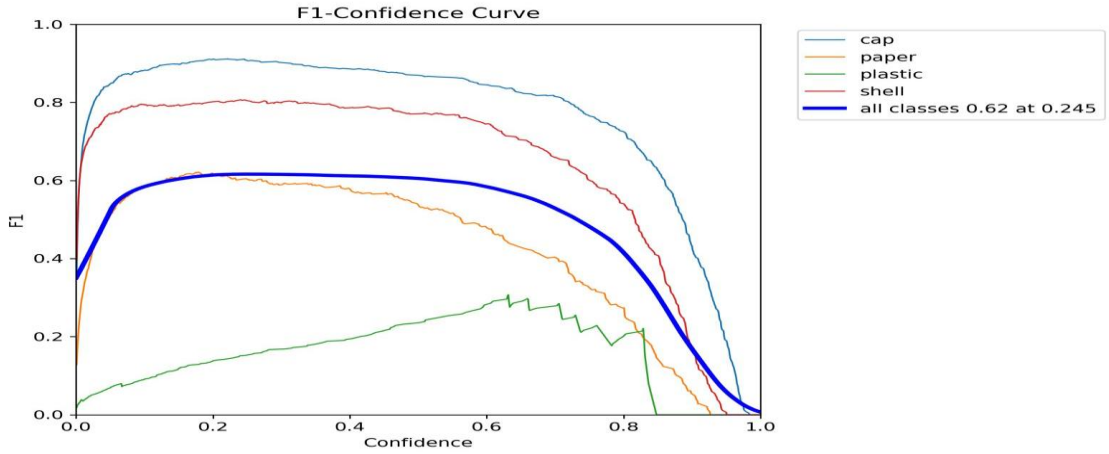
SCREENSHOTS OF OUTPUTS :

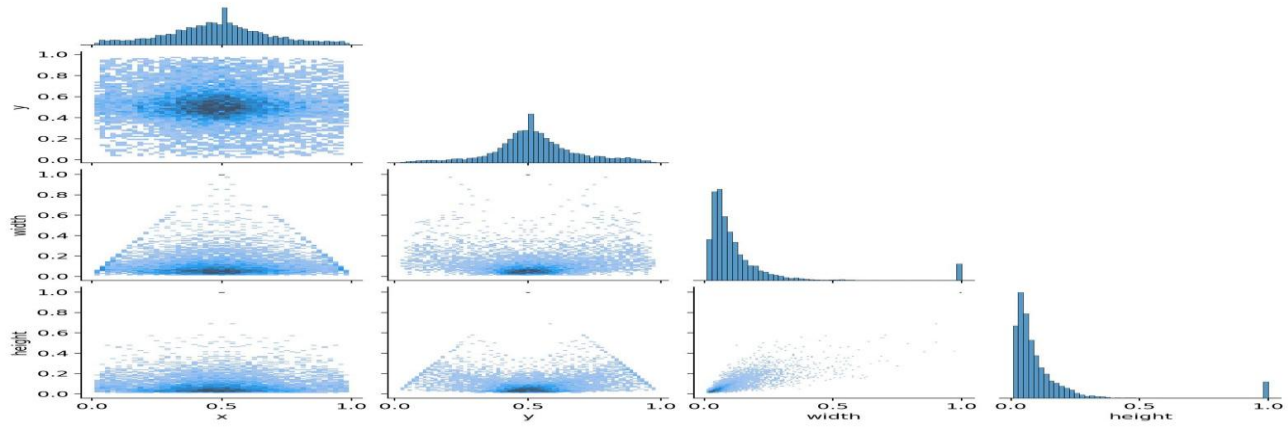
SYNTHETIC IMAGE GENERATED USING GAN :

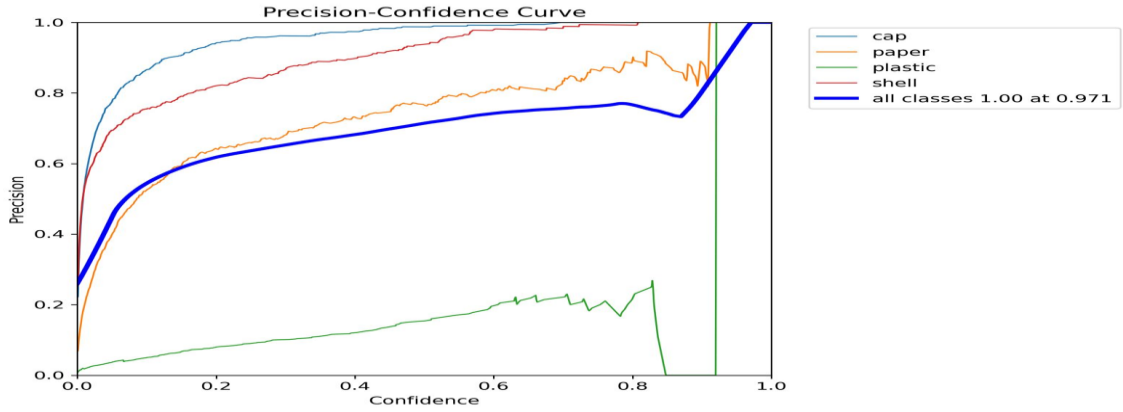


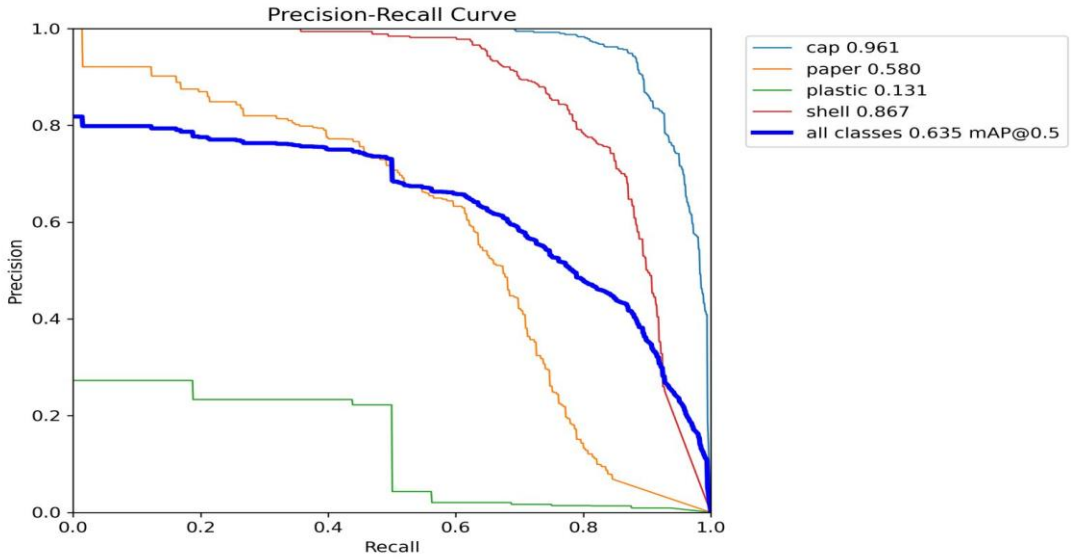
YOLO V8:

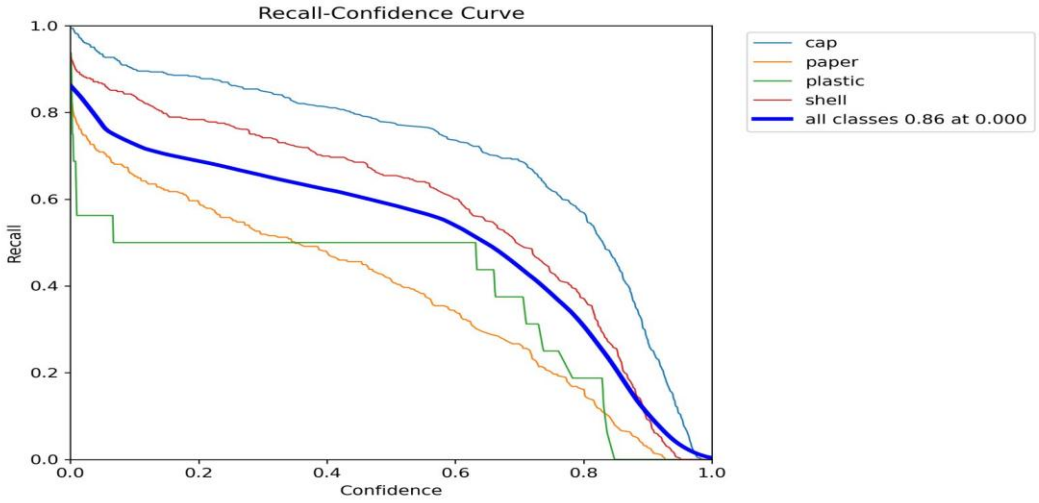


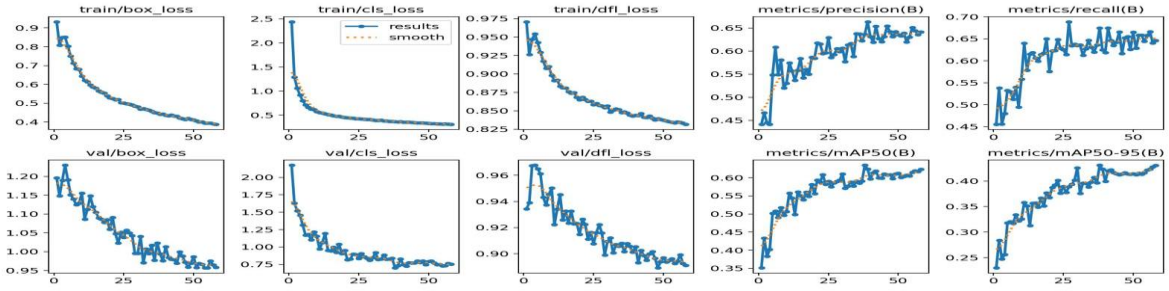

F1-Confidence Curve
cap
paper
plastic
shell
all classes 0.62 at 0.245
F1
Confidence


x
y
width
height


Precision-Confidence Curve
cap
paper
plastic
shell
all classes 1.00 at 0.971
Precision
Confidence


Precision-Recall Curve
Precision
Recall
cap 0.961
paper 0.580
plastic 0.131
shell 0.867
all classes 0.635 mAP@0.5

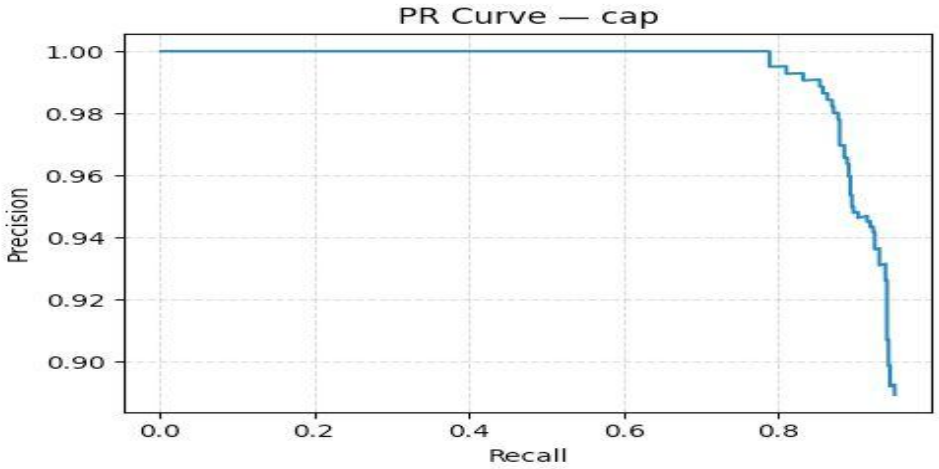

Recall-Confidence Curve
Recall
Confidence
cap
paper
plastic
shell
all classes 0.86 at 0.000

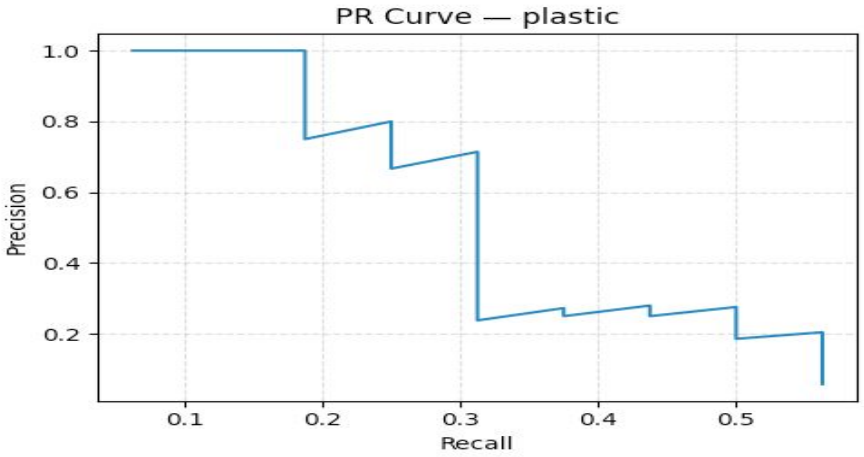

train/box_loss
train/cls_loss
results
smooth
train/dfl_loss
metrics/precision(B)
metrics/recall(B)
val/box_loss
val/cls_loss
val/dfl_loss
metrics/mAP50(B)
metrics/mAP50-95(B)

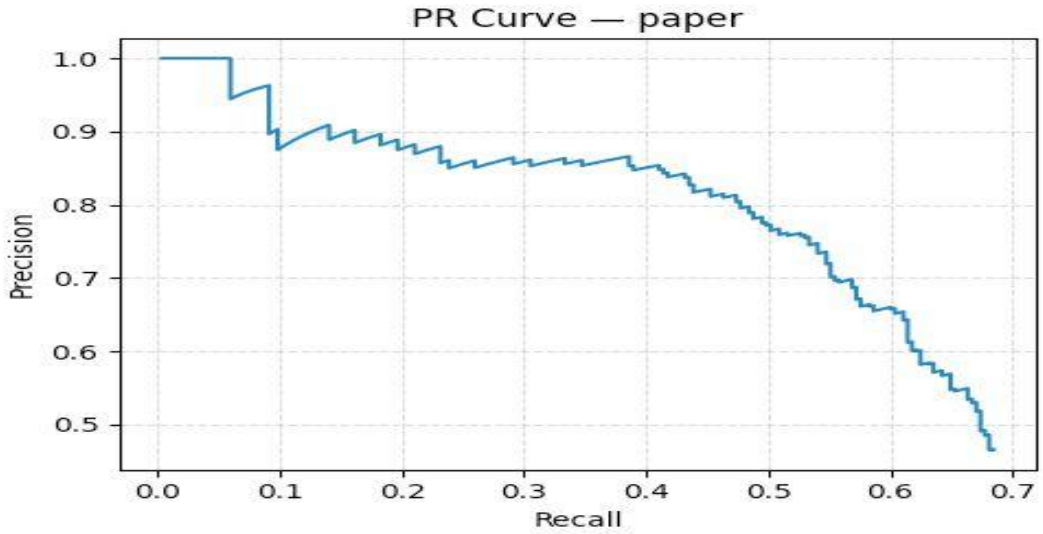

cap
plastic

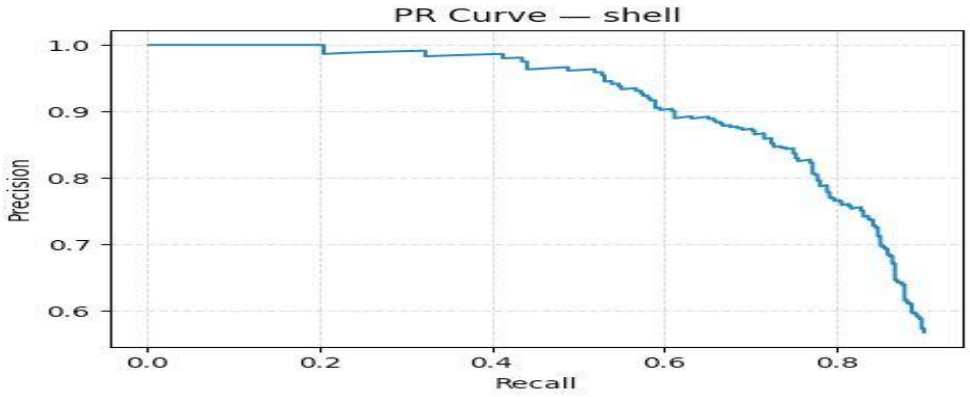

paper
plastic

R-CNN :

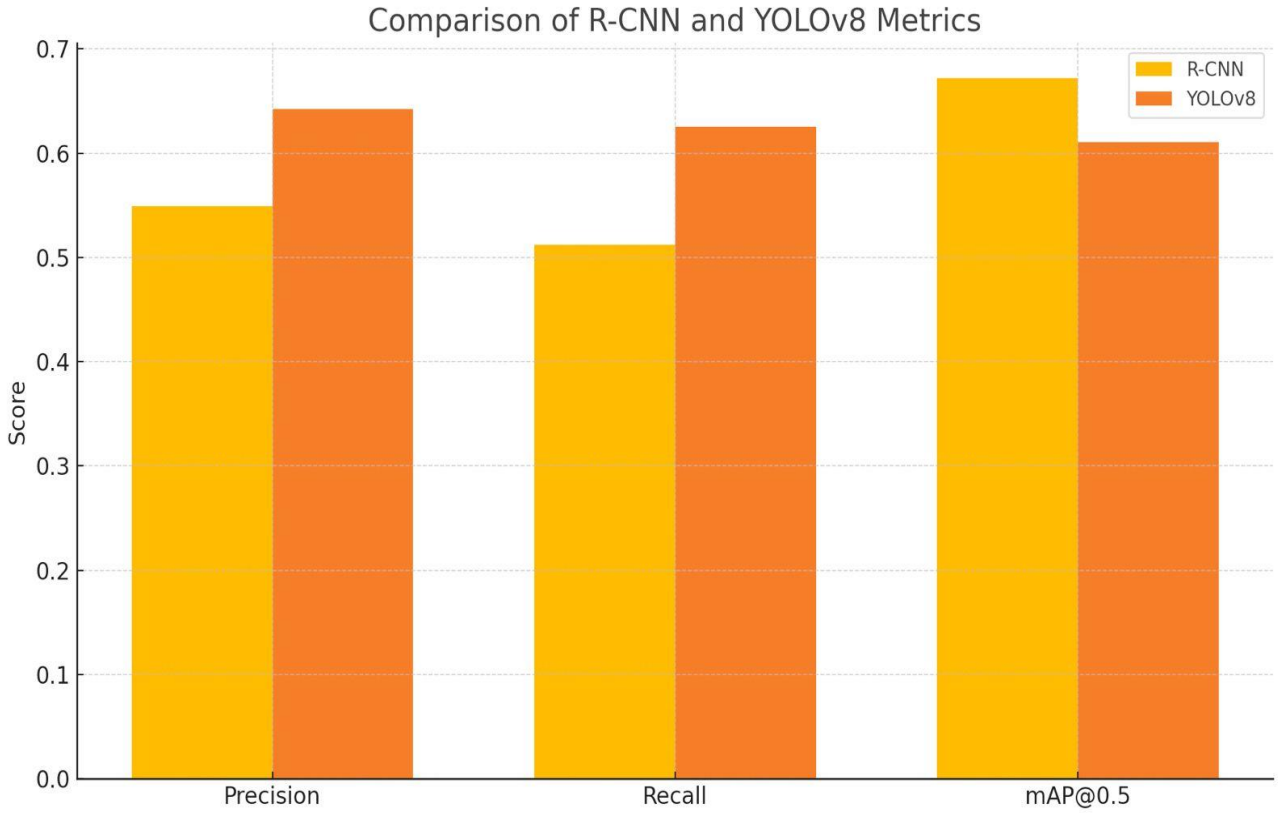

PR Curve — cap
Precision
Recall
1.00
0.98
0.96
0.94
0.92
0.90
0.0
0.2
0.4
0.6
0.8


PR Curve — plastic
Precision
Recall
1.0
0.8
0.6
0.4
0.2
0.1
0.2
0.3
0.4
0.5


PR Curve — paper
Precision
Recall
1.0
0.9
0.8
0.7
0.6
0.5
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7



COMPARISON CHARTS :



Model	mAP@0.5	Inference Speed (FPS)	Model Size (MB)
YOLOv8	0.6104	41.7	6.28
R-CNN	0.6723	5.3	178.3

REFERENCE :

[1] Lun, Z., Pan, Y., Wang, S. et al. Skip-YOLO: Domestic Garbage Detection Using Deep Learning Method in Complex Multi-scenes. Int J Comput Intell Syst 16, 139 (2023). https://doi.org/ 10.1007/s44196-023-00314-6.

[2] Ren Y, Li Y, Gao X. An MRS-YOLO Model for High-Precision Waste Detection and Classification. Sensors (Basel). 2024 Jul 4;24(13):4339. doi: 10.3390/s24134339. PMID: 39001117; PMCID: PMC11244501.

[3] Sylwia Majchrowska, Agnieszka Mikołajczyk, Maria Ferlin, Zuzanna Klawikowska, Marta A. Plantykow, Arkadiusz Kwasigroch, Karol Majek,Deep learning-based waste detection in natural and urban environments,Waste Management,Volume 138,2022,Pages 274-284,ISSN 0956-053X, https://doi.org/10.1016/j.wasman.2021.12.001.

[4] Md. Wahidur Rahman, Rahabul Islam, Arafat Hasan, Nasima Islam Bithi, Md. Mahmodul Hasan, Mohammad Motiur Rahman,Intelligent waste management system using deep learning with IoT,Journal of King Saud University - Computer and Information Sciences,Volume 34, Issue 5, 2022,Pages 2072-2087,ISSN 1319-1578,https://doi.org/10.1016/j.jksuci.2020.08.016.

[5] Gupta, T., Joshi, R., Mukhopadhyay, D. et al. A deep learning approach based hardware solution to categorise garbage in environment. Complex Intell. Syst. 8, 1129–1152 (2022). https:// doi.org/10.1007/s40747-021-00529-0.

[6] Jin, S., Yang, Z., Królczykg, G., Liu, X., Gardoni, P., & Li, Z. (2023). Garbage detection and classification using a new deep learning-based machine vision system as a tool for sustainable waste recycling. Waste Management, 162, 123-130.

[7] Sarswat, P. K., Singh, R. S., & Pathapati, S. V. S. H. (2024). Real time electronic-waste classification algorithms using the computer vision based on convolutional neural network (cnn): Enhanced environmental incentives. Resources, Conservation and Recycling, 207, 107651.

[8] Zhang, Q., Zhang, X., Mu, X., Wang, Z., Tian, R., Wang, X., & Liu, X. (2021). Recyclable waste image recognition based on deep learning. Resources, Conservation and Recycling, 171, 105636.

[9] Rahmatulloh, A., Darmawan, I., Aldya, A. P., & Nursuwars, F. M. S. (2025). WasteInNet: Deep Learning Model for Real-time Identification of Various Types of Waste. Cleaner Waste Systems, 10, 100198.

[10] Adedeji, O., & Wang, Z. (2019). Intelligent waste classification system using deep learning convolutional neural network. Procedia Manufacturing, 35, 607-612.

[11] Jiang, X., Hu, H., Qin, Y. et al. A real-time rural domestic garbage detection algorithm with an improved YOLOv5s network model. Sci Rep 12, 16802 (2022). https://doi.org/10.1038/s41598-022-20983-1

[12] Jiang, X., Hu, H., Qin, Y. et al. A real-time rural domestic garbage detection algorithm with an improved YOLOv5s network model. Sci Rep 12, 16802 (2022). https://doi.org/10.1038/s41598-022-20983-1

[13] Zailan Nur Athirah , Azizan Muhammad Mokhzaini , Hasikin Khairunnisa , Mohd Khairuddin Anis Salwa , Khairuddin Uswah ,An automated solid waste detection using the optimized YOLO model for riverine management,Frontiers in Public Health,10,2022,https://www.frontiersin.org/journals/public-health/articles/10.3389/fpubh.2022.907280,DOI=10.3389/fpubh.2022.907280 ISSN=2296-2565

[14] Liu S, Chen R, Ye M, Luo J, Yang D, Dai M. EcoDetect-YOLO: A Lightweight, High-Generalization Methodology for Real-Time Detection of Domestic Waste Exposure in Intricate Environmental Landscapes. Sensors (Basel). 2024 Jul 18;24(14):4666. doi: 10.3390/s24144666. PMID: 39066064; PMCID: PMC11280945.

[15] Qiang Zhang, Qifan Yang, Xujuan Zhang, Wei Wei, Qiang Bao, Jinqi Su, Xueyan Liu,A multi-label waste detection model based on transfer learning,Resources, Conservation and Recycling,Volume 181,2022,106235,ISSN 0921-3449,

[16] Kumar, S., Yadav, D., Gupta, H., & Kumar, M. Smart e-waste management system utilizing Internet of Things and Deep Learning approaches. Journal of Smart Cities and Society, 2(2), 77-92 (2023).

[17] Chen, W.-T., Lin, Y.-C., Chen, Y.-H., & Tsai, C.-H. Deep learning networks for realtime regional domestic waste detection. Journal of Cleaner Production, 367, 132728 (2022).

[18] Wang, X., Zhang, Y., & Liu, Z. Garbage Detection Algorithm Based on YOLO v3. In 2022 IEEE 6th Information Technology and Mechatronics Engineering Conference (ITOEC) (pp. 352-357). IEEE (2022).

[19] Adedeji, O., & Wang, Z. YOLO TrashNet: Garbage Detection in Video Streams. In 2020 IEEE International Conference on Artificial Intelligence and Computer Applications (ICAICA) (pp. 1126-1130). IEEE (2020).

[20] Liu, Z., Zhang, J., Li, X., & Wang, Q. Solid Waste Detection Using Enhanced YOLOv8 Lightweight Convolutional Neural Networks. Mathematics, 12(14), 2185 (2024).

CONCLUSION

This project successfully implemented and compared two state-of-the-art object detection models, YOLOv8 and R-CNN, for automatic waste detection in indoor public places. The key findings include:

1. Both models demonstrate strong performance in detecting and classifying four common waste categories (paper, plastic, cap, and shell)
2. R-CNN achieves higher detection accuracy (mAP@0.5 of 0.6723) but at the cost of slower inference speed (5.3 FPS)
3. YOLOv8 provides significantly faster detection (41.7 FPS) while maintaining good accuracy (mAP@0.5 of 0.6104), making it more suitable for real-time applications
4. Data augmentation and synthetic data generation using GANs significantly improve model performance, particularly for underrepresented classes
5. The choice between YOLOv8 and R-CNN depends on the specific requirements of the deployment scenario, with YOLOv8 being more appropriate for real-time applications and R-CNN for scenarios where accuracy is paramount

The project demonstrates the effectiveness of deep learning approaches for waste detection and provides valuable insights into the trade-offs between different models for practical deployment.